Prof. Dr. Joachim Sauer

NORDAKADEMIE
HOCHSCHULE DER WIRTSCHAFT

Probeklausur Softwaretechnik (I151)

Name des Prüflings: **Matrikelnummer:** **Zenturie:**

Dauer: 90 Min. Seiten der *Klausur* **ohne** Deckblatt: 15 Datum: 27. September 2016

Hilfsmittel:

- NORDAKADEMIE Taschenrechner

Bemerkungen:

- **Bitte prüfen Sie zunächst die Klausur (alle Teile) auf Vollständigkeit.**

Anzahl Antwortseiten einzelnes kariertes Zusatzpapier: [ ] (vom Prüfling auszufüllen)

Es sind 100 Punkte erreichbar.
Zum Bestehen der Klausur sind 50 Punkte ausreichend.

Aufgabe	1	2	3	4	5	6	Summe
Erreichbare Punkte	6	12	16	15	7	44	100
Erreichte Punkte							

Note: _____ Prozentsatz: _____ Ergänzungsprüfung: _____

Datum: _____ Unterschrift: _____

Datum: _____ Unterschrift: _____

Aufgabe 1 (6 Punkte)

SOFTWARETECHNIK

Bewerten Sie den Wahrheitsgehalt der folgenden Aussagen zur Softwaretechnik:

Aussage	Wahr	Falsch
Die Hauptkosten der Softwareentwicklung liegen in der Hardwareausstattung und Lizenzkosten. (Das sind Wartungskosten nach Release)		X
Die Komplexität der Softwareentwicklung hat in den letzten Jahrzehnten abgenommen.		X
Die Gebrauchsqualität einer Software ergibt sich aus ihrer internen Qualität.		X
Als Legacy Software bezeichnet man neue Anwendungssysteme, bei denen sich Fehler im ersten Einsatz zeigen.		X
Die Disziplin Softwaretechnik entstand mit Einführung der ersten objektorientierten Systeme in den 1980er Jahren. (-> 1968 von F.L. Bauer)		X
Die organisatorische Planung der Softwareentwicklung gehört nicht zur Disziplin Softwaretechnik. doch ∇ siehe Planungsphase		X

Hinweis zur Punktevergabe: 1 Punkt pro richtigem Kreuz; 1 Punkt Abzug pro falschem Kreuz. Die gesamte Aufgabe wird mit mindestens 0 Punkten bewertet.

Aufgabe 2 (12 Punkte)
VORGEHENSMODELLE

(2.1) (6 Punkte) Skizzieren Sie das Wasserfallmodell und erläutern Sie die einzelnen Phasen mit ihren wichtigsten Ergebnisartefakten.

Das Wasserfallmodell nutzt das Resultat einer Phase als Input für die nächste Phase -> es entsteht eine Einbahnstraße.

Planung: Lastenheft
↳ Definition: Pflichtenheft
↳ Entwurf: Architekturen & Entwurfsmuster
↳ Implementierung: Source-Code
↳ Einführung: Tests, Dokumentation
↳ Wartung: Bugfixing

(2.2) (6 Punkte) Nennen und diskutieren Sie drei wichtige prinzipielle Unterschiede zwischen plangetriebenen und agilen Vorgehensmodellen.

1. Vorgehen: Im plangetriebenen Modell entsteht ein Einbahnstraßenmodell: Dort werden meistens die einzelnen Phasen iterativ durchlaufen und der Input ist immer stark abhängig aus der vorherigen Phase. Agil nutzt Teilprodukte und dort kann man zu vorigen Phasen jederzeit zurückkehren.

2. Qualität & Kosten Verhältnis: plangetrieben: Fokus auf Kosten und deren Optimierung. Zudem soll die eigene Funktionalität der Software im Vordergrund stehen
Agiles Vorgehensmodell konzentrieren sich auf die Qualität.

3. Einbeziehen der Stakeholder: Während bei der klassischen Methode nur am Anfang und Ende ausgewählte Stakeholder einbezogen werden, versucht man im agilen alle Stakeholder einzubeziehen.

Aufgabe 3 (16 Punkte)

Planung und Definition

(3.1) (4 Punkte) Worin unterscheiden sich Lastenheft und Pflichtenheft? Nennen Sie zwei wesentliche Unterschiede.

Definition genauer Aufgabenzuordnungen:
- Im Lastenheft nur grobe Ziele und Erwartungen (Probleme aus Kundensicht)
- Im Pflichtenheft gibt es dann genaue Aufgaben, die bearbeitet werden müssen.

Erwartungen im Lastenheft.

Keine Erwartungen mehr im Pflichtenheft, dafür genaue Anforderungen.

(3.2) (4 Punkte) Wofür steht die Abkürzung LoC? Wofür lässt sich die LoC-Methode einsetzen?

LoC = Lines of Code

Sie sind ein einfaches Mittel zur Aufwandsschätzung.

Über diese lässt sich ein erster, einfacher Schluss über die Größe eines Projekts ziehen, nicht aber über Qualität oder tatsächliche Komplexität.

(3.3) (4 Punkte) Beschreiben Sie die ersten beiden Schritte der Function-Point-Methode mit ihrer jeweiligen Zielsetzung und grenzen Sie sie voneinander ab.

1. Kategorisierung

2. Klassifizierung

Anforderungen benötigen (1) um z.B. als technisch, organisatorisch u.ä. definiert zu werden.

(2) beschreibt z.B. den Umfang durch Klassen wie „einfach", „Mittel" oder „komplex".

(3.4) (4 Punkte) Wie kommt man von einem Function-Point-Wert zu einer Schätzung der Dauer eines Projekts? Was ist die wichtigste Voraussetzung für sinnvolle Ergebnisse?

Man greift immer auf Erfahrungswerte, also empirische Daten zurück, um FP-Werte konkreten Zeiteinheiten zuzuordnen.

Dabei müssen die Erfahrungswerte vorhanden sein und das Projekt sollte seine Daten auch immer in den Erfahrungsschatz zurückfließen lassen!!

Aufgabe 4 (15 Punkte)
OO-Designprinzipien

(4.1) (6 Punkte) Nennen Sie vier Symptome für schlechtes Design und beschreiben Sie sie jeweils in einem Satz.

- Komplexität: Lösungen sind nicht einfach gehalten
- Zerbrechlichkeit: Änderungen im Code können das Projekt zerstören
- Undurchsichtigkeit: Der Code kann schwer gelesen und verstanden werden.
- Wiederholungen: Änderungen können durch redundante Codeabschnitte das Projekt zerstören.

(4.2) (3 Punkte) Erläutern Sie das Prinzip der Strukturähnlichkeit.

Strukturähnlichkeit bezeichnet die Ähnlichkeit von Klassen, Objekten, Methoden usw. Sie können leicht wiederverwendet werden und sind einfacher in der Pflege, besser erweiterbar und leichter verständlich.

(4.3) (6 Punkte) Beschreiben Sie die Blutgruppen von Quasar und erläutern Sie, wie sie beim Entwurf von Klassen helfen.

A: Beschreiben Abhängigkeiten durch die Anwendung

T: ... durch Technik

AT: ... durch Anwendung und Technik

O: sind unabhängig

R: transformieren fachliche Objekte in z.B. lesbare Objekte (speicherbar, darstellbar)

Die Blutgruppen können wunderbar Zuständigkeiten aufzeigen und bei deren Trennung helfen!

↳ indem Klassen auf die verschiedenen Blutgruppen aufgeteilt werden.

Aufgabe 5 (7 Punkte)
SOFTWAREARCHITEKTUR

(5.1) (4 Punkte) Nennen Sie eine Definition von Softwarearchitektur, die alle wesentlichen Eigenschaften umfasst.

„[Softwaretechnik ist ...] die Struktur eines Softwareproduktes. Diese umfassen Elemente (Komponenten), die extern wahrnehmbaren Eigenschaften der Elemente und die Beziehungen zwischen den Elementen."

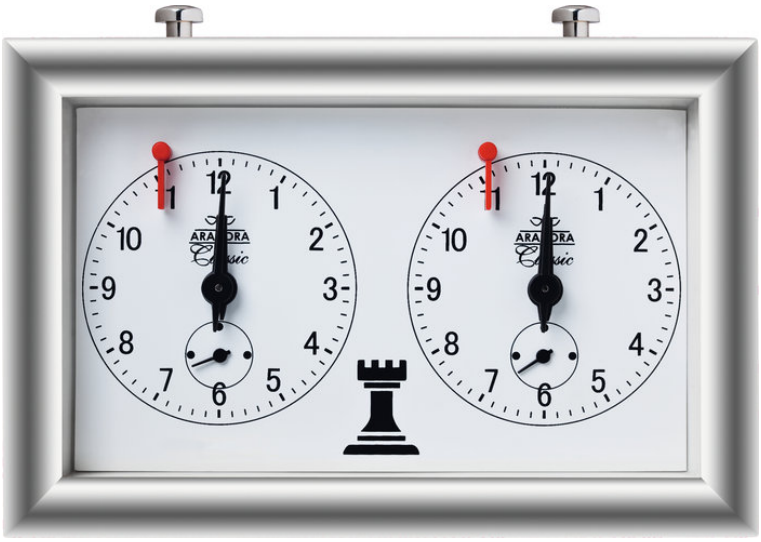
(5.2) (3 Punkte) Erläutern Sie, warum unterschiedliche Sichten auf die Softwarearchitektur eines Systems sinnvoll sind.

Da sie verschiedene Anforderungen abdecken und unterschiedliche Aspekte begutachten und sich um diese kümmern.
=> fachlich, statisch, Laufzeit, Verteilung

Aufgabe 6 (44 Punkte)

UML-Modellierung

Ihr Unternehmen soll eine Software entwickeln, die eine Schachuhr umsetzt, so wie sie in der Abbildung dargestellt wird. Sie sind damit beauftragt, einen ersten Entwurf in UML-Diagrammen zu erstellen. Lesen Sie zunächst gründlich folgenden Text zum Kontext, bevor Sie sich an die Modellierung in den Teilaufgaben machen. Achten Sie darauf, dass die von Ihnen erstellten Diagramme alle wesentlichen Informationen beinhalten und untereinander konsistent sind.



Im professionellen Schachspiel wird eine Schachuhr zur Zeitmessung in einer Schachpartie eingesetzt. Schach wird von zwei Spielern – Weiß, der immer beginnt, und Schwarz – gespielt, indem abwechselnd Züge auf dem Schachbrett ausgeführt werden. Jeder Spieler hat eine bestimmte Zeit zum Nachdenken und zur Ausführung seiner Züge zur Verfügung. Ein Spieler gewinnt, wenn der andere Spieler matt gesetzt wird, aufgibt oder seine Zeit überschreitet. Die Spieler können sich auch auf ein Unentschieden, Remis genannt, einigen. (In der Realität gibt es noch weitere Fälle, aber dies sind die wesentlichen.) Ein Schiedsrichter überwacht den Spielablauf.

Eine Schachuhr besteht aus zwei Uhren, eine für Weiß, die andere für Schwarz, welche die Zeit anzeigen, die dem jeweiligen Spieler für die Partie verbleibt. Es läuft immer nur eine Uhr zur Zeit. Mit einer Taste auf der Seite von Weiß wird die weiße Uhr angehalten und die schwarze in Gang gesetzt und umgekehrt.

Vor Beginn der Partie stellt der Schiedsrichter die Bedenkzeit für beide Spieler auf ihrer jeweiligen Uhr ein. Anschließend drückt er die Taste auf der schwarzen Seite, sodass die Zeit des weißen Spielers läuft. Nachdem Weiß seinen Zug ausgeführt hat, drückt dieser die Taste auf seiner Seite. Dann ist Schwarz an der Reihe, führt seinen Zug aus und drückt die Taste auf seiner Seite. Jetzt ist wieder Weiß am Zug. Ein kleines Blättchen in jeder Uhr (in der Abbildung bei 11 Uhr) wird vom Uhrzeiger angehoben, wenn sich die Zeit dem Ende neigt. Sobald es fällt, gilt die Zeit des betroffenen Spielers als abgelaufen

und er verliert die Partie. Es ist Aufgabe des Schiedsrichters, auf den Zeitablauf zu achten und den Sieger zu bestimmen. Wenn keiner der Spieler durch Zeitablauf verliert, läuft die Partie, bis einer der Spieler auf andere Art und Weise gewinnt oder sich die Spieler auf Remis einigen. Der Schiedsrichter hält abschließend die Uhr an.

Um Kosten zu sparen, soll eine vorhandene Klasse benutzt werden, die eine Uhr mit *einem* Ziffernblatt zur Verfügung stellt. Es gibt Operationen, um die Uhr einzustellen, um sie zu starten und anzuhalten sowie um die noch verbleibende Zeit abzulesen.

Zeichnen Sie die Diagramme unter Angabe der Aufgabennummer auf den folgenden Leerseiten.

(6.1) (8 Punkte) Erstellen Sie ein *Anwendungsfalldiagramm* für Ihre Schachuhr-Software, die alle Akteure und die wesentlichen Anwendungsfälle zeigt, die sich aus dem Beschreibungstext ergeben.

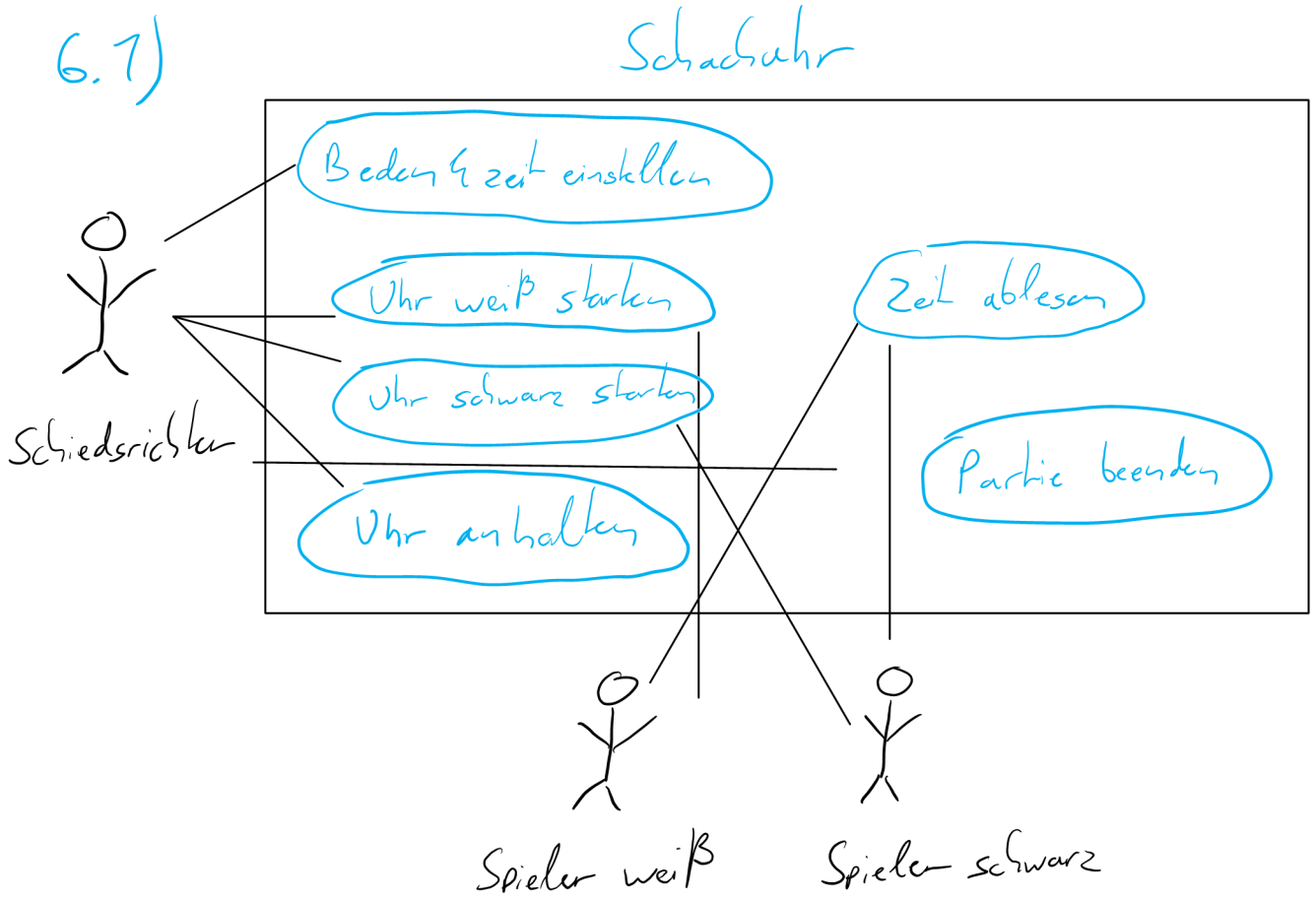
(6.2) (8 Punkte) Erstellen Sie ein *Zustandsdiagramm* für die Schachuhr, das mit dem Einstellen der Uhr durch den Schiedsrichter beginnt und alle möglichen Ausgänge der Partie zeigt.

(6.3) (8 Punkte) Erstellen Sie ein *Klassendiagramm*. Dieses soll alle wesentlichen Klassen der Software beinhalten. Auch die vorhandene Uhren-Klasse ist darzustellen. Wie oben beschrieben, ist diese Uhrenklasse zwingend zu benutzen. Geben Sie für alle Klassen die wichtigsten Attribute und sinnvolle Operationen an.

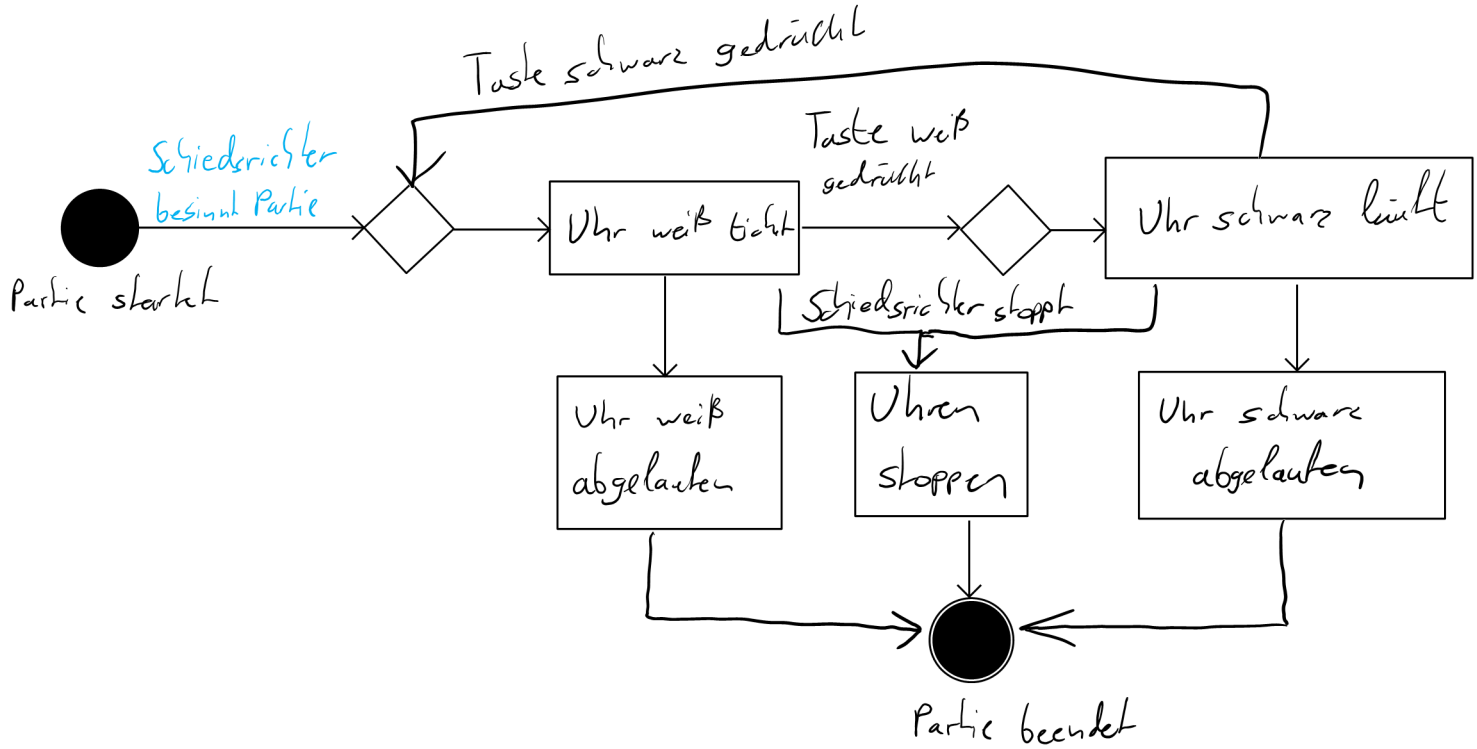
(6.4) (6 Punkte) Erstellen Sie ein *Objektdiagramm*, das die Situation nach Spielstart und einigen Zügen von Weiß und Schwarz zeigt. Weiß ist am Zug und hat noch 50 Minuten Zeit übrig. Schwarz verfügt noch über ein Zeitguthaben von 55 Minuten. Geben Sie für alle Objekte die momentanen Belegungen ihrer Attribute an. Die Operationen können Sie weglassen, da sie schon aus dem Klassendiagramm hervorgehen.

(6.5) (14 Punkte) Erstellen Sie ein *Sequenzdiagramm*, das eine Partie vom Einstellen der Uhren durch den Schiedsrichter über den Spielstart, abwechselnde Züge von Weiß und Schwarz (jeweils einer reicht, verwenden Sie Auslassungszeichen) bis zum Ende der Partie zeigt, die Weiß durch Zeitüberschreitung verliert. Stellen Sie dabei sicher, dass sich alle an der Schachuhr und ihren Komponenten aufgerufenen Operationen in Ihrem Klassendiagramm wiederfinden lassen.

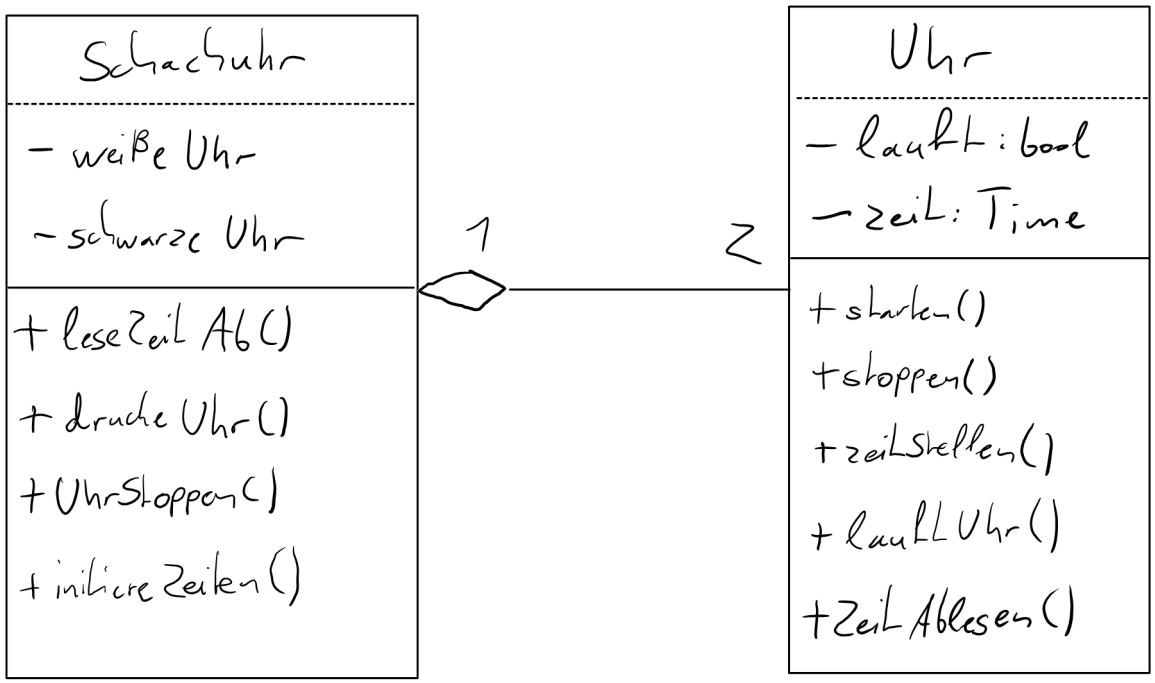
6.1)


Schachuhr
Bedenkzeit einstellen
Uhr weiß starten
Uhr schwarz starten
Uhr anhalten
Zeit ablesen
Partie beenden
Schiedsrichter
Spieler weiß
Spieler schwarz

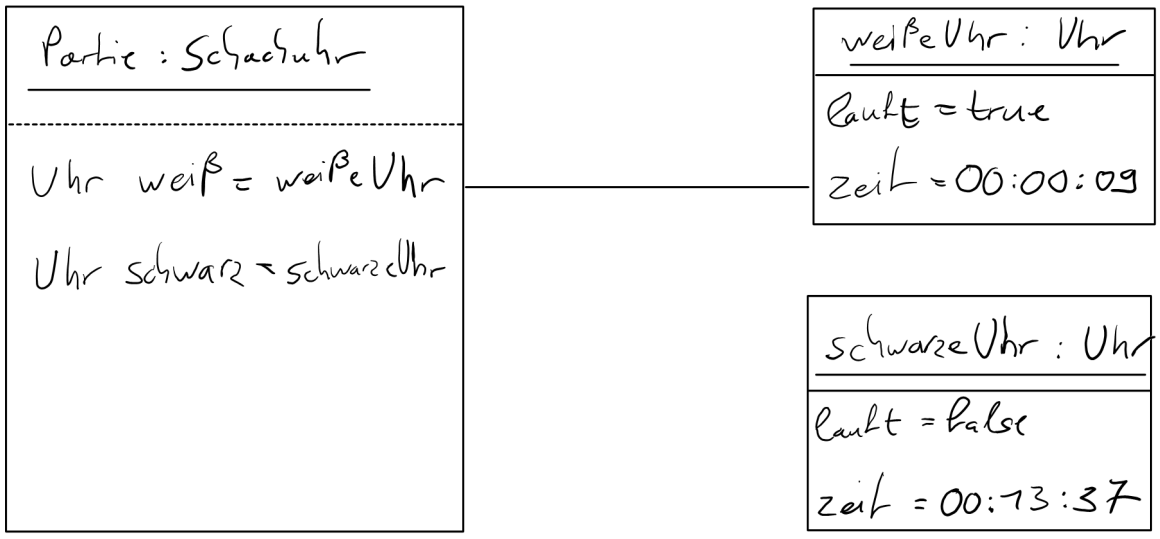
6.2)


Taste schwarz gedrückt
Taste weiß gedrückt
Schiedsrichter beginnt Partie
Partie startet
Uhr weiß läuft
Uhr schwarz läuft
Schiedsrichter stoppt
Uhr weiß abgelaufen
Uhren stoppen
Uhr schwarz abgelaufen
Partie beendet

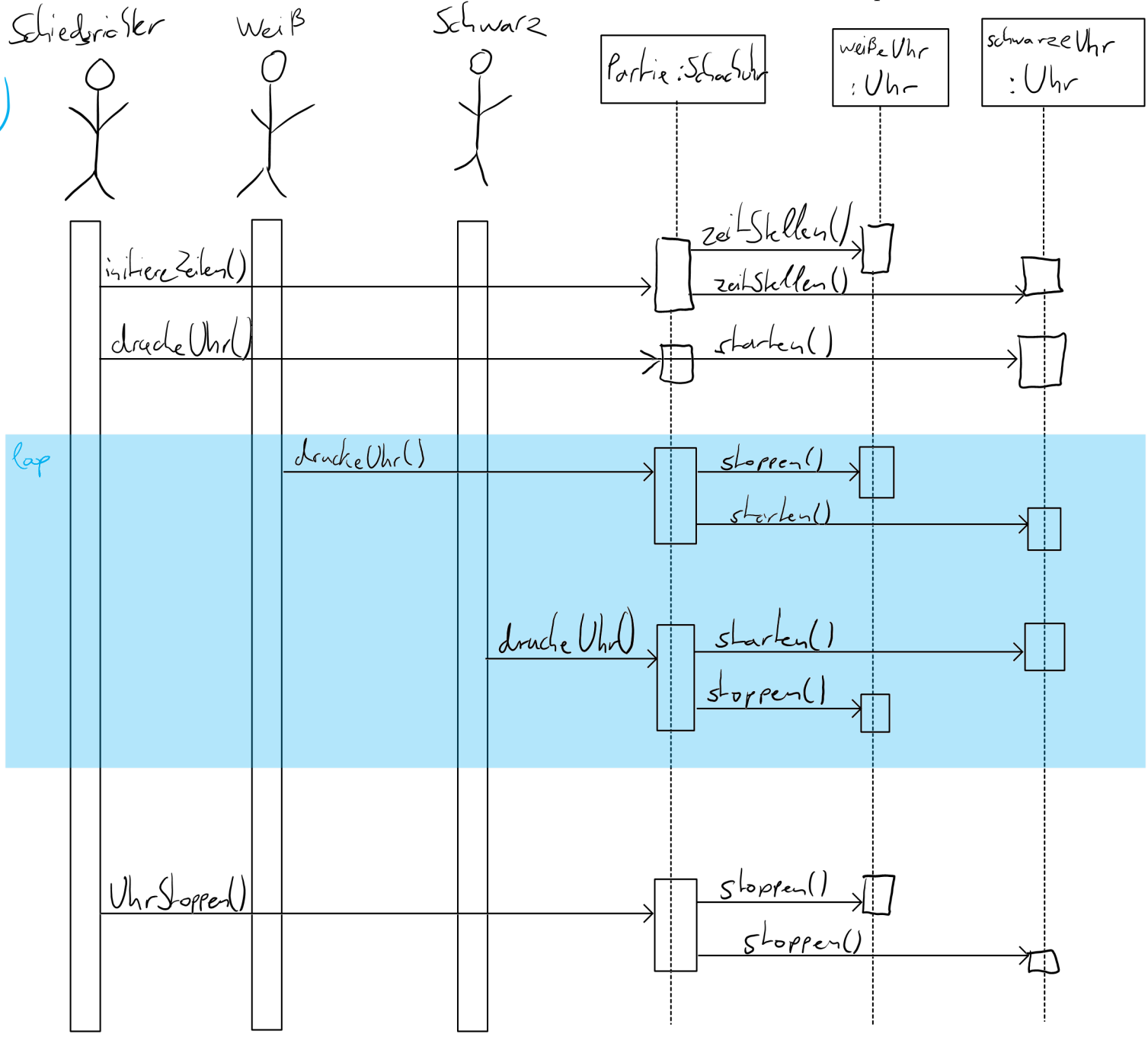
6.3)


Schachuhr
- weiße Uhr
- schwarze Uhr
+ leseZeitAb()
+ druckeUhr()
+ UhrStoppen()
+ initiiereZeiten()
1
2
Uhr
- laeuft: bool
- zeit: Time
+ starten()
+ stoppen()
+ zeitStellen()
+ laufLUhr()
+ ZeitAblesen()

6.4)


Partie : Schachuhr
Uhr weiß = weißeUhr
Uhr schwarz = schwarzeUhr
weißeUhr : Uhr
laeuft = true
zeit = 00:00:09
schwarzeUhr : Uhr
laeuft = false
zeit = 00:13:37

6.5)


Schiedsrichter
Weiß
Schwarz
Partie:Schachuhr
weiße Uhr
:Uhr
schwarze Uhr
:Uhr
initiiereZeiten()
zeitStellen()
zeitStellen()
drucheUhr()
starten()
loop
drucheUhr()
stoppen()
starten()
drucheUhr()
starten()
stoppen()
UhrStoppen()
stoppen()
stoppen()